Problem Reconstruction

Conversational Memory Intelligence System

Deliverable 1 · AI Engineering Cohort · Author: Ketan Juikar · June 2026

This document explains why a memory system has to exist, before any design is proposed. I have deliberately written the problem without naming the solution. Someone who has never seen my final design should be able to read this and explain, in their own words, why the system is needed, which simpler designs fail, and what those failures demand.

I have spent twenty years in enterprise systems, mostly .NET and SQL Server at Capgemini and BNP Paribas, and more recently on production machine learning for fraud detection. My instinct — and my actual job as an AI catalyst — is to make a system justify itself before it gets built. So that is how I am approaching this. Not "here is a memory architecture," but "here is a cost we keep paying, and here is what it would take to stop paying it."

1. The problem

By default, an assistant treats every request on its own. It answers the message in front of it and keeps nothing. Across a relationship that spans many sessions, that means the user has to set up the same context again and again: who they are, what they are working on, what has already been decided, and how they want to be spoken to.

A concrete example from my own use. I am running a long .NET-to-Java migration with an assistant. In one session I explain the code conventions, the placement rules, and the fact that there are no existing tests to lean on. In the next session, none of that survives. I explain it again. The assistant cannot honour a single thing I told it the day before.

The same gap shows up in my own product, CommentHook. An assistant that forgets a user's business context between sessions is barely more useful than a search box.

What fails: information learned in one conversation is lost before the next one. **Who it affects:** anyone in an ongoing relationship with the assistant, and most of all people working on long projects. **When:** whenever the value depends on continuity, rather than on one self-contained question.

2. Why it matters

Leaving this unsolved has four consequences, and none of them are cosmetic.

- **Wasted user effort.** The human pays the cost of setting up context on every session. The work is purely mechanical, which is exactly the kind of thing that should be automated away.
- **Wasted money.** If context is supplied by pasting the whole history back in, the same tokens get paid for again and again, and the bill grows as the relationship gets longer and more valuable.
- **No personalisation.** Without remembered preferences, the assistant cannot adapt to how a particular person wants to be helped.
- **Lost trust.** An assistant that forgets — or worse, remembers wrongly — loses the user's confidence quickly, and that is hard to win back.

3. Constraints

Any real solution has to live inside hard limits. These are the ones I will hold the design to.

Constraint	What it means here
Token budget	The model's context window is limited and you pay for it. Whatever memory we inject must fit inside a fixed budget, alongside the actual conversation.
Speed	Memory sits on the live request path. Looking it up should take under about 100 milliseconds, so it does not visibly slow the answer down.
Cost	Every write and every lookup costs money. The system has to stay affordable as the store grows.
Privacy	Conversations contain sensitive data. Secrets and personal information must be kept out of the store, and deletion must be real and checkable.
Correctness	A confidently wrong memory is worse than no memory. The system must know the difference between what it is sure of and what it is guessing.
Keeping users separate	In any multi-user system, one person's memories must never reach another. This is a security boundary, not a nice-to-have.

4. Simpler approaches, and why they fail

A few simpler designs look like they might solve this. Each one makes an assumption that breaks in real use. I summarise them here. The full chain is in `historical_timeline.pdf`, and the detailed failure cases are in `failure_analysis.md`.

Approach	What it quietly assumes	Where it breaks
Remember nothing	Every question is self-contained	Nothing survives the session, so the original problem is untouched

(stateless)		
Replay the whole history every time	Context window, cost, and speed are effectively free	Overflows the window; cost and speed get worse as the relationship grows
Store everything, search by similarity	Storing everything is fine, because search will filter it	Junk buries the good memories; old facts win over new ones; exact names get missed; users are not separated; nothing is ever forgotten

5. Evidence for the failures

The failures are written up as specific cases in `failure_analysis.md`, each marked by how strong the evidence is: **measured** (to be proven with numbers in the baseline experiment), **assumed** (a grounded thought experiment), or **cited** (backed by a paper).

I am writing the failures down before building the baseline on purpose. If the baseline does not reproduce them, that is itself a finding, and it changes the design. The main cases are: junk burying good memories, outdated facts winning, exact names being missed, one user seeing another user's data, secrets being kept, and the store growing forever.

6. How the field got here

Nobody arrived at managed memory in one step. The field moved through a sequence, each step forced by the limit of the one before it. Remembering nothing gave way to replaying everything, which hit the context window wall and gave way to external stores and search. That hit problems with junk and relevance, and gave way to selective, typed, governed memory. The full chain, with the research that marks each step, is in `historical_timeline.pdf`.

7. What the system therefore has to do

From the failures, a set of required capabilities follows by necessity, not by preference. They are derived in full in `first_principles.md`. In short, the system must decide whether to write anything at all, store proper typed records, find them using several signals, settle contradictions, forget on a schedule, keep users separate and protect their data, watch itself, survive its own outages, and be measurable against a baseline. Every one of these traces back to a specific failure, not to a wish list of features.

> **The governing idea.** Memory is not storage. A database stores everything. A memory system stores only what will improve a future answer. So the first thing standing in front of the store is not a write — it is a decision about whether to write at all. Everything else in the design follows from that one idea.

8. Open questions

Some decisions are deliberately left open here. They belong in the system design, once the baseline experiment has produced real numbers.

- When a passing remark should be promoted into a lasting memory, and on how much evidence.
- Whether the write gate runs on the live request path or in the background, and when it should stop being an AI judge and become a small trained classifier.
- How the ranking weights get set and tuned honestly, without a labelled set of correct answers.
- How fast a deleted memory must disappear from search.
- Which separation strategy fits: row-level security inside one database, or physically separate storage. The answer for a small product and for a regulated bank may not be the same.

References (full list in the project bibliography): Vaswani et al., *Attention Is All You Need* (2017); Lewis et al., *Retrieval-Augmented Generation* (2020); Park et al., *Generative Agents* (2023); Packer et al., *MemGPT* (2023).